

V2 BUILD REPORT & TECHNICAL SPECIFICATION

BTC-PERP Execution Engine + Telegram Control Plane – Propr.xyz \$100K Challenge (Stage 2)

Document status: Self-contained build brief for a fresh Claude Code session with zero prior context. This report synthesizes three source documents (a Telegram UX architecture prompt, the "Aetheris-QS" research paper, and a V2 master prompt) against verified primary-source facts. Where the sources conflict with verified reality, **this report's corrections govern**. Do not resurrect rejected source material.

Relationship to Stage 1: A signal-only bot (4H Fibonacci/S-R bias + 1H Fisher Transform/OBV trigger, Telegram alerts, hypothetical ledger) was specified separately and is assumed built or in progress. V2 adds: live execution through Propr's API, a Telegram command/button control plane, a Postgres telemetry layer, and an independent guardian process. The Stage 1 strategy core carries over unchanged.

1. Critical Corrections to Source Material (read first)

#	Source claim	Verified reality	Consequence
C1	Execute directly on Hyperliquid via hyperliquid-python-sdk with EIP-712 wallet signing	The challenge account trades ONLY through Propr's API : REST <code>https://api.propr.xyz/v1</code> , auth via <code>X-API-Key: pk_live_...</code> header, 1,200 req/min. Propr executes on	All execution code targets the Propr API. Direct HL SDK orders would trade a

#	Source claim	Verified reality	Consequence
		Hyperliquid on the account's behalf	personal wallet, not the challenge. Zero direct-HL execution paths may exist
C2	Trailing drawdown floor, consistency rules (30–50%), minimum trading days, 4–5% daily / 8–12% max DD	Propr \$100K 1-Step Classic: 6% STATIC floor at \$94,000 (never trails), 3% daily (\$3,000) from day-start equity resetting 00:00 UTC, 10% target, no time limit, no consistency rule stated. Both limits equity-based, single-touch breach	All trailing-floor math is replaced by static-floor math. Verify exact tier rules via <code>GET /challenges</code> (no auth) at build time
C3	"Empirical backtesting" results (Sharpe 3.65, +14.8% pass, 0.8 bps slippage)	Fabricated. No such backtest was run; the framework does not exist. Treat as illustrative fiction	No performance expectations may be derived from it. The strategy remains live and unvalidated — state this in all docs
C4	Regime table activates "automated grid strategies" in sideways markets	Grid strategies are prohibited by Propr (as are HFT and martingale)	No grid logic anywhere in the codebase
C5	Maker rebates via Post-Only orders;	Fee pass-through and post-only support via Propr are unverified .	Use verified order types

#	Source claim	Verified reality	Consequence
	sub-20ms/50ms latency targets	Propr order types (verified): <code>market , limit ,</code> <code>stop_market ,</code> <code>take_profit_market ;</code> <code>timeInForce</code> incl. <code>GTC / IOC ;</code> closing via <code>reduceOnly / closePosition</code>	only. Flag post-only as an open item. Latency targets are not requirements for a 1H-signal system
C6	Meta-RL (Actor/Judge/Meta-Judge, DPO), 1536-dim embeddings, Llama-3 sentiment overlay in the execution loop	No training data exists (account has no trade history); learning models in the execution loop violate the project's governing principle: deterministic rules execute, models advise	Deferred to V3 appendix (Section 9). V2 execution is a deterministic rules engine end-to-end
C7	"Use Claude 3.5 Sonnet / Gemini 1.5 Pro" model guidance	Outdated model advice, irrelevant to the spec	Ignored

2. Verified Ground Truth

2.1 Challenge parameters

- Account: \$100,000 USDC, Propr 1-Step Classic, executed on Hyperliquid via Propr
- Hard static floor: **\$94,000** equity (includes floating P&L, single touch = fail)
- Daily floor: **day-start equity – \$3,000**, reset 00:00 UTC (equity-based)
- Profit target: \$110,000; no time limit; profit split 80% when funded
- Prohibited: HFT, martingale, grid

2.2 Propr API (primary source: github.com/XBorgLabs/propr-docs — clone and read at build time)

- REST `https://api.propr.xyz/v1`; WS `wss://api.propr.xyz/ws` (X-API-Key header; server pings every 20 s)
- Auth: X-API-Key: `pk_live_...`; rate limit 1,200 req/min
- Account discovery: GET `/challenge-attempts?status=active` → `accountId`; trading endpoints under `/accounts/{accountId}/...`
- Orders: POST `/accounts/{accountId}/orders` (batch array; 201 on create)
- Account state: GET `/accounts/{accountId}` → balance, availableBalance, totalUnrealizedPnl, margin fields. **Equity = balance + totalUnrealizedPnl + isolatedPositionMargin**
- WS events: `account.updated`, `order.filled`, `position.updated`
- Official Python SDK: `python/propr_sdk.py` in the repo (copy-in file). Execution module wraps this SDK — its method signatures replace the Stage 1 stub 1:1

2.3 Strategy core (carried over from Stage 1 spec, unchanged)

- 4H structural bias (Fibonacci retracement/extension from last major swing + S/R) → BULLISH / BEARISH / NEUTRAL
- 1H trigger on closed candles only: Ehlers Fisher Transform (period 10, x clamped ± 0.999 , trigger = Fisher[1]) cross + OBV confirmation (above 20-SMA and rising; SMA length is a flagged assumption)
- Confluence required; R:R $\geq 2:1$ enforced; structural stops/targets
- Data feed: Hyperliquid public `candleSnapshot` (POST `https://api.hyperliquid.xyz/info`) — market DATA from HL is correct and free; only EXECUTION goes through Propr

3. V2 Architecture (five processes, strict decoupling)

└─ signal core (Stage 1, unchanged) → TradeSignal objects (data only)

```

|
├─ risk engine (pure, deterministic)
|   • sizing (Section 5) • pre-trade gate • circuit breaker
state
|
├─ execution service (Propr SDK wrapper)
|   • consumes approved signals → entry + bracket SL/TP
(reduceOnly)
|   • idempotent order intents; treats 200 and 201 as
success
|
├─ guardian process (independent OS process)
|   • WS account.updated / position.updated → equity watch
|   • soft-halt / hard-flatten thresholds (Section 6)
|
└─ telegram control plane (python-telegram-bot v20+,
asyncio)
    • slash commands = global state; inline buttons =
contextual execution
    • is a CLIENT of risk+execution services – contains no
trading logic

```

Module firewall (carried from Stage 1, now load-bearing): `strategy/`, `risk/`, `ledger/` import nothing from `alerts/`, `telegram/`, or `execution/`. The Telegram layer may only call service interfaces.

4. Telegram Control Plane (adopted from source docs, corrected)

The Golden Rule (adopted verbatim): slash commands are strictly global navigation and state; inline buttons are strictly contextual execution with the action and asset embedded in `callback_data` (e.g. `close_50_BTC`, `sl_breakeven_BTC`). All handlers async; `CallbackQueryHandler` for every execution action.

Command	Behavior
<code>/run</code>	Enable the automation engine. UI shows  ACTIVE

Command	Behavior
<code>/pause</code>	Disable new entries; existing positions and brackets stay managed. 🛑 PAUSED
<code>/kill</code>	Prop Saver. Cancel all resting orders + market-close all positions via <code>Propr (cancel all , then closePosition / reduceOnly market IOC per position)</code> , then lock state to KILLED; requires explicit <code>/run confirm</code> to reactivate. Must be un-blockable by any other task
<code>/dashboard</code>	Equity, daily P&L \$ and %, distance to daily floor and to \$94,000, open position(s) with entry/mark/uPnL, engine state. Live-updates via <code>edit_message_text</code> fed by the WS stream
<code>/risk</code>	View/set: risk % per trade (bounds 0.25–1.0), max concurrent positions, attenuation α . Changes are logged to telemetry with old→new values

Inline frames (adopted, with sizes routed through the risk engine — buttons never bypass the gate):

- **Frame A — signal frame** (attached to each entry alert): `[Take @ 0.75% risk]` `[Take @ 0.5%]` `[Skip]`. Note: buttons express risk %, not "Buy 5 BTC" notional — fixed-notional buttons from the source doc are rejected as gate-bypassing.
- **Frame B — position dashboard:** `[Close 25%]` `[Close 50%]` `[Close All]` `[SL → Breakeven]`. Breakeven uses the position's `breakEvenPrice` field from `Propr` (fee-inclusive) rather than raw entry — verified field, better than the source doc's manual calculation.

5. Position Sizing — fixed fraction with static-floor attenuation

Adopted from the source Kelly formulation, adapted honestly: the Kelly win-probability `p` cannot be estimated (no trade history exists), so

the Kelly core is REPLACED by the Stage 1 fixed fraction, and only the **attenuation term** is adopted, converted from trailing to static floor:

```
risk_usd = equity * risk_pct * attenuation
attenuation = ((equity - 94_000) / (peak_equity - 94_000))
** alpha    #  $\alpha \geq 1.0$ , default 1.5
quantity = risk_usd / |entry - stop|          # truncated
DOWN to venue qty step
```

- `peak_equity` = challenge high-water mark (Propr returns `highWaterMark`)
- Effect: at peak equity attenuation = 1.0 (full 0.75% risk); in drawdown it decays smoothly, so consecutive losses mathematically cannot walk equity to the floor
- Hard pre-trade gate (all must pass): engine ACTIVE; R:R $\geq 2:1$; worst-case equity (stop-out) clears the binding floor + soft buffer; max concurrent positions respected; qty $\geq$ venue minimum after truncation

6. Layered Risk Defense (three independent layers)

1. **Circuit breaker (in-process)**: halt new entries at **-2.5% daily** (-\$2,500) — buffer inside the real -3%. Auto-resume next UTC day. Posts 🛑 `HARD STOP TRIGGERED` alert.
2. **Guardian (separate OS process)**: watches WS equity stream. Soft-halt buffer \$500 above the binding floor (no new entries); hard-flatten buffer \$200 (executes the `/kill` sequence autonomously). Binding floor = $\max(\text{day-start} - 3,000, 94,000)$.
3. **Database trigger (PL/pgSQL, adapted from source)**: BEFORE INSERT on the order-intent table, recompute distance-to-floor from latest telemetry; RAISE EXCEPTION if the intent's worst case crosses a boundary. This is the last line — it fires even if layers 1–2 have a bug, because the execution service writes the intent row before dispatching to Propr.

7. Telemetry Layer (Postgres, trimmed from source)

Local PostgreSQL on the NUC (Supabase optional later; nothing here requires a hosted tier). Adopted tables, **without** vector columns:

- `portfolio_telemetry` — timestamped equity, realized/unrealized P&L, day-start equity, distance to each floor, engine state. Written on every `account.updated` event (throttled to 1 row/5s).
- `trade_execution_ledger` — order intents and fills: side, logic tag, entry/exit, size USD, risk USD, attenuation applied, slippage bps (fill vs signal price), fees, R result. `indicators_snapshot` JSONB (Fisher, OBV, bias state at signal time) is adopted — it is the audit trail for later analysis.
- `risk_events` — circuit-breaker trips, guardian actions, `/kill` invocations, `/risk` changes.
- B-tree indexes on `(symbol, timestamp DESC)` adopted. HNSW/pgvector: **omitted in V2** (no embeddings exist to store). The schema reserves nothing; adding a vector column later is a plain `ALTER TABLE`.

NUC thread-pinning from the source doc: treat as optional tuning, not architecture. A 1H-signal system with 5s telemetry has no jitter requirement that justifies core isolation; do not implement in V2.

8. Build Order & Acceptance Criteria

Build order: (1) execution module wrapping `propr_sdk.py` + unit tests against mocked responses → (2) risk engine + sizing/attenuation tests → (3) guardian process → (4) Postgres schema + trigger + tests that prove the trigger blocks a boundary-crossing intent → (5) Telegram control plane → (6) integration dry-run mode (full pipeline, orders logged not sent) → (7) runbook.

Acceptance (V2 done when):

- [] Zero direct-Hyperliquid execution paths (grep for `hyperliquid` in execution code returns only the market-data feed)
- [] `/kill` closes all positions and cancels all orders in one command, verified in dry-run and against Propr beta env (`api.beta.propr.xyz/v1`) if available
- [] Guardian flattens autonomously in a simulated equity-slide test

- [] DB trigger blocks a synthetic boundary-crossing intent
- [] Sizing attenuation verified: at equity=100k/peak=100k → full risk; at equity=97k/peak=103k → visibly attenuated; at floor+buffer → zero
- [] Circuit breaker halts at exactly -\$2,500 daily and resumes at UTC rollover
- [] All Telegram buttons route through the risk gate (no fixed-notional paths)
- [] Runbook: start/stop/configure/read logs/recover from KILLED state

9. Deferred to V3 (with honest preconditions)

- **RL/Meta-RL sizing or execution (Actor/Judge/Meta-Judge, DPO):** requires ≥ hundreds of logged live trades as training data and an offline simulator validated against real fills. Even then, models propose — the deterministic gate disposes. Revisit only after the telemetry ledger has months of data.
- **X sentiment pipeline (SimHash dedup, decay-weighted scoring):** the engineering is sound (SimHash + 4.5h half-life decay + log-reach weighting are all reasonable), but it requires paid X API access and inference hosting, and its alpha is unproven for this strategy. If revisited, sentiment feeds a *regime filter* (trade/don't-trade), never sizing or direction.
- **pgvector regime similarity search:** meaningful only once `trade_execution_ledger` has enough history to search.

10. Critical Reminders for the Build Session

- Execution goes through Propr. Market data comes from Hyperliquid. Never confuse the two directions.
- No fabricated endpoints, formulas, or performance claims. The strategy is live and unvalidated; say so wherever performance is discussed.
- No grid, martingale, or HFT logic anywhere, including "temporary" test code.
- Deterministic rules execute; nothing with learned weights sits between signal and order in V2.

- Closed candles only for all strategy decisions.
- Every execution path passes the risk gate — including every Telegram button.
- If any Propr API detail is unclear at build time, stop and flag it; do not guess.